Deep Learning Systems — Analysis Report

Project: Capstone Project 4 (Udacity AI Mastery) **Dataset:** CNRPark (original, not EXT) — http://cnrpark.it/ **Architecture:** 3-block CNN (baseline) vs. 4-block CNN (experimental) with the same `Flatten` →FC head

---

0. Report overview

This project addresses **binary image classification** on parking-lot camera patches, predicting for each 150×150 RGB image whether a parking space is `busy` (occupied) or `free` (empty). The dataset is **CNRPark** (Amato et al., ISTI-CNR, 2016) — a publicly released research corpus of 12,584 patches from two cameras, used here for educational comparison under the ISTI-CNR terms. The deep-learning model implemented is a **Convolutional Neural Network (CNN)** in PyTorch, with two configurations compared: a three-block baseline and a four-block experimental variant that differs by exactly one architectural change (one additional convolution block) so the comparison isolates that single lever.

Plain-English walkthrough (read this first if you're not a deep-learning engineer)

This section explains the whole project without any jargon. Every technical term that appears later in the report is defined here in plain language, so you can read the rest with confidence.

What we built, and why

Imagine a security camera watching a parking lot. Every few seconds it captures a small picture of one parking space. A human security analyst can look at each picture and instantly tell whether the space is occupied (a car is parked there) or empty. We wanted a computer to do the same job — look at a picture and answer **"busy"** or **"free"** — so the analyst doesn't have to.

The computer program we built to do this is called a **Convolutional Neural Network**, or **CNN** for short. A CNN is a kind of machine-learning model loosely inspired by the visual cortex: it learns to recognize patterns in small patches of an image (edges, corners, blobs of color), then combines those patterns into bigger structures (a wheel, a bumper, a shadow), and eventually outputs one decision ("busy" or "free"). You don't tell it what to look for — you show it thousands of labeled pictures and it figures out the patterns itself. For a tutorial-level overview of the convolution operation and the canonical CNN building blocks used here, see O'Shea & Nash (2015).

For this project, we built **two** CNNs and compared them. They differ by exactly one architectural choice (more on that below) so the comparison tells us whether that choice actually helped or hurt.

What data we used

We used the **CNRPark** dataset, published for free by an Italian research institute (ISTI-CNR). It contains **12,584 small images** of parking spaces, each labeled either `busy` (a car is there) or `free` (empty). The images come from two different cameras — call them **Camera A** and **Camera B** — and are roughly twice as likely to show a busy space as a free one (because busy spaces are common in real parking lots).

We split those 12,584 images into three groups:

Group	Size	Purpose
Training set	10,068 images (80%)	The model studies these and learns from them.
Validation set	1,258 images (10%)	We peek at this during training to decide when to stop and to compare model designs.
Test set	1,258 images (10%)	We **never look at this until the very end.** It's the model's final exam — the score on it is the headline number we report.

Holding out a test set we never trained on is the standard way to make sure the model has actually *learned* something general, rather than just memorized the answers to the questions it was asked during practice.

What the two CNNs look like, and how they differ

Both CNNs read a 150×150 RGB image and produce one number: a "score" between 0 and 1, where high means "busy" and low means "free". Inside, they're built out of small building blocks stacked on top of each other:

- A **convolution block** scans the image with many tiny filters (e.g. 3×3 pixels) and produces a slightly smaller picture of "what it found" — early blocks find edges and colors, later blocks find larger structures. Each block typically doubles the number of filters while halving the spatial size via max-pooling.
- A **fully-connected (FC) head** at the end takes all those findings and combines them into the final score.

The two CNNs are:

- **BaselineCNN** — three convolution blocks (32 → 64 → 128 filters), then a standard FC head. About **5.4 million** parameters (the adjustable knobs the model tunes during

training).
- **ExperimentalCNN** — **one additional** convolution block (the 128-filter block is followed by a 256-filter block), then the same kind of FC head. About **3.0 million parameters.**

The single difference is **depth** — one extra convolution block. Both end with the same kind of decision-making head, so any difference in how well they learn is attributable to that one extra block.

Why the experimental model is smaller, not bigger

You might expect "more layers = more parameters". In this case, the opposite happens, and it's worth understanding why. After the conv blocks, the baseline model flattens a **128-channel × 18×18-pixel** picture into a single long vector (about **41,500 numbers**) and feeds it to the FC head. The experimental model, with one extra MaxPool operation that halves the spatial size, flattens a **256-channel × 9×9-pixel** picture (about **20,700 numbers**) — fewer numbers to feed to the head, so the head itself has fewer parameters. The extra convolution block adds about 280k parameters; the smaller head removes about 2.7M. Net change: the experimental model is **1.8× smaller** overall. This is a well-known modern-CNN pattern: depth + spatial shrinking can be more parameter-efficient than shallow-and-wide.

What the headline numbers mean in plain language

We trained each model for **20 passes over the training data** (called "epochs") on a single NVIDIA GTX 1650 Ti GPU. After training, we evaluated each model on the test set — the 1,258 images it had never seen. Here is what we got:

Model	Test accuracy	What that means in everyday terms
BaselineCNN (3 blocks)	**99.76%**	Out of 1,258 test images, it got **1,255 right** and **3 wrong.**
ExperimentalCNN (4 blocks)	**98.49%**	Out of 1,258 test images, it got **1,239 right** and **19 wrong.**

Both are good, but on this single run the baseline is clearly ahead — by about **16 extra mistakes per 1,258 images**. There is a catch, though: the experimental model's final training epoch was unstable (more on this below), so its test score partly reflects a bad final moment rather than its typical performance. The multi-seed comparison below (which stops training earlier, before the instability) is the fairer test.

Why we ran the experiment 3 times with different random seeds

When you train a neural network, you start it with **random initial weights** (the starting guess for all those knobs) and the training process is partly random — the order in which the model

sees the training images is shuffled each epoch. Different random starts produce slightly different final models, just like different chefs making the same recipe get slightly different meals.

To find out whether the experimental model is *really* different from the baseline (or whether one was just lucky on this particular run), we retrained both models with **three different random seeds** (7, 42, 99) and compared the results.

The paired comparison (each seed gives a baseline number AND an experimental number, so we look at the *difference* under each seed) tells us:

Seed	Baseline test acc	Experimental test acc	Difference
7	99.52%	99.13%	−0.40 pp (baseline wins)
42	98.73%	98.65%	−0.08 pp (baseline wins)
99	99.44%	98.65%	−0.79 pp (baseline wins)
Mean	**99.23%**	**98.81%**	**−0.42 pp (baseline wins on average)**

Three observations from this table:

1. **The baseline model wins on the multi-seed average** (+0.42 percentage points, or about 5 fewer mistakes per 1,258 images).
2. **The baseline wins on all 3 individual seeds** — the deeper model never beat it.
3. **The deeper model is slightly more consistent across seeds** (std 0.28 pp vs 0.44 pp for the baseline) — but "consistently worse" is not a win. The smaller variance comes from the deeper model reliably landing in a narrower (lower) accuracy band.

What we can and cannot claim from this experiment

We can claim:

- Both CNNs solve the parking-occupancy problem well — above 98.5% accuracy on the held-out test set.
- The deeper (4-block) CNN uses **about half as many parameters** (3.0M vs 5.4M).
- On this dataset, **the deeper model did not help** — the baseline wins on the single 20-epoch run (+1.27 pp), wins on the multi-seed mean (+0.42 pp), and wins all 3 individual seeds.

We cannot claim:

- That "deeper is always worse" for image classification. This is one specific dataset; other datasets and tasks may behave differently.
- That the deeper model is *useless*. Its `best_val_acc` (0.9968) ties the baseline — it can reach baseline-level performance; it is just less stable late in training and weaker on one

camera.
- That depth alone is the right architectural lever for all problems. We tested **one** alternative (depth). Other alternatives (BatchNorm, global average pooling, attention) might matter more. The deeper model's late-epoch instability suggests a learning-rate schedule or early stopping might recover the gap.

What this has to do with the larger capstone (P7)

The capstone's final project is a security-operations copilot that ingests camera events and access logs, scores them as potential incidents, and alerts an analyst. The pipeline includes both classical ML (sklearn) and modern AI components. Our parking-space classifier here is the **simplest possible surveillance event**: a binary "is something there?" decision from one image. It demonstrates the deep-learning pattern end-to-end, and the lesson that transfers to P7 is **methodological** — when you change a model, change *one thing at a time* so you know what helped and what didn't.

A note on honesty

Throughout this report we report **real numbers** and call out exactly which claims are well-supported by the data and which are tentative. The single-run accuracy difference between baseline and experimental (1-2 mistakes out of 1,258) is too small to draw conclusions from; the multi-seed comparison is the load-bearing evidence. We do not have enough seeds (3) to claim the difference is statistically significant; a larger follow-up would tighten this.

---

1. Problem statement

Binary occupancy classification on parking-lot camera patches. Given a 150×150 RGB image, predict whether the parking space is `busy` (occupied, label 1) or `free` (empty, label 0). The model that solves this well is the simplest possible surveillance-camera event classifier — relevant to P7's incident-fusion layer (a "space-just-became-busy" event is exactly what an occupancy CNN would emit).

1.1 Why CNRPark

CNRPark (Amato et al., ISTI-CNR, 2016) is a clean binary image-classification dataset: ~12.5k 150×150 patches across two cameras (A, B), labeled `busy`/`free` by folder. It is small enough to train a CNN on a laptop in minutes per epoch, large enough that the model has to learn rather than memorize, and natural for the capstone's security-ops theme. The published baseline in the Amato paper reports ≈99% accuracy on this dataset, so the question is not "can we solve it?" but "does a deeper model improve on a shallower one under controlled conditions?"

1.2 License

Research / educational use only (ISTI-CNR terms). Dataset excluded from version control via `.gitignore`. The trained models are not deployed in any real surveillance system.

---

2. Data

2.1 On-disk summary

camera	busy (1)	free (0)	total
A	3,621	2,550	6,171
B	4,781	1,632	6,413
total	**8,402**	**4,182**	**12,584**

- Class imbalance: ~2:1 toward `busy`; `B/busy` is the dominant class.
- Patch resolution: 150×150 native (no resize — code keeps it to avoid distortion).
- Split (SEED=42): train = 10,068 (80%), val = 1,258 (10%), test = 1,258 (10%).
- Same split indices for both baseline and experimental — the only difference between runs is the model.

2.2 Preprocessing

- `Resize(150, 150) → ToTensor → Normalize(mean=ImageNet, std=ImageNet)` shared by train and eval. The ImageNet-mean/std normalization is the standard preprocessing used with PyTorch's pretrained vision backbones (Paszke et al., 2019); using it here keeps the option open of warm-starting from pretrained weights in a follow-up.
- Training augmentation: `RandomHorizontalFlip` + `RandomVerticalFlip`. **Shared by both runs** so the comparison isolates the architectural change, not augmentation.
- No color jitter: would fight the weather/lighting signal we want the model to see.

2.3 Companion metadata (not used in this report)

`data/CNRPark+EXT.csv` (157,550 rows) contains per-frame metadata (camera, datetime, weather, occupancy, slot_id, image_url) for richer ablation experiments. Flagged as a follow-up; the baseline-vs-experiment question is answerable from the patches alone.

---

3. Methods

3.1 Baseline model — `BaselineCNN`

- 3 conv blocks, doubling channels per block: `Conv(3→32) → ReLU → MaxPool(2)` → `Conv(32→64) → ReLU → MaxPool(2)` → `Conv(64→128) → ReLU → MaxPool(2)`.

Implemented with `nn.Conv2d` + `nn.MaxPool2d` (PyTorch `torch.nn` API; Paszke et al., 2019).
- Feature map at the end of the conv stack: `128 × 18 × 18` (downsampled 8× from the 150×150 input).
- Head: `Flatten → Linear(128·18·18, 128) → ReLU → Linear(128, 1)`. The first linear layer is the parameter hog (~5.3M params out of 5.4M total).
- **Total parameters: 5,401,921.**
- No normalization, no dropout, no regularization beyond the shared training augmentation.

3.2 Experimental model — `ExperimentalCNN`

- One architectural change vs. baseline: **one extra conv block** (`Conv(128→256) → ReLU → MaxPool(2)`) added after the third block.
- Feature map at the end of the conv stack: `256 × 9 × 9` (downsampled 16× from the input).
- Head: **same** `Flatten → Linear(256·9·9, 128) → ReLU → Linear(128, 1)` mechanism. The input dim to the first linear grows because the wider feature map is wider per channel but smaller in spatial extent; the net effect is a **smaller head** (`Linear(20736, 128)` instead of `Linear(41472, 128)`).
- **Total parameters: 3,042,881** — about **1.8× smaller** than baseline.
- No BatchNorm, no dropout, no GAP. The head type and the conv-block mechanism are identical to baseline; only depth changes.

3.3 Why depth as the one change

- **Cleanest possible isolated architectural change.** The head mechanism (`Flatten`→FC) is the same on both sides. The two features (deeper stack + wider feature map) are coupled by design — adding a conv block without a smaller head would balloon the FC layer's parameter count.
- **Defensible in defense.** A reviewer can read the diff in notebook cell 13 line-by-line and see exactly the 4th conv block as the change.
- **Real test of depth** rather than a regularization knob (BatchNorm) or a head-mechanism change (GAP).

3.4 Training procedure

- **Loss:** `BCEWithLogitsLoss` (the numerically stable pairing of a sigmoid + binary cross-entropy — PyTorch docs, Paszke et al., 2019). **Optimizer:** `Adam(lr=1e-3)` (Kingma & Ba, 2015). **Batch size:** 64. **Epochs:** 20 (single-seed §4) / 10 (multi-seed §5).
- `DEVICE = cuda if available else cpu` in `src/train.py` — resolves to `cuda:0` on the GTX 1650 Ti. CUDA tensor semantics and `torch.device` handling follow the PyTorch documentation (Paszke et al., 2019).

- **Seeding:** `SEED = 42` global constant in `src/dataset.py` (data split) and `src/train.py` (default). Multi-seed runs vary the model's `seed=` argument only; the data split is held constant.
- **DataLoaders:** `num_workers = 2` (Windows-safe via the `build_dataloaders` function pattern). `DataLoader` shuffling and batching semantics follow the PyTorch documentation (Paszke et al., 2019).
- Per-epoch metrics persisted to `reports/metrics_*.csv` by notebook cell-19a so Restart & Run All is reproducible.

3.5 Hardware / software

- `cnn_env` Python 3.12 venv at `D:/AI_Master/Udacity/capstone_projects/project_04_DeepLearning/cnn_env/`
- `torch==2.12.1+cu126`, `torchvision==0.27.1+cu126` (CUDA build; NVIDIA GTX 1650 Ti, driver CUDA 13.1). PyTorch tensor and `torch.nn` APIs used throughout (Paszke et al., 2019).
- `pandas`, `matplotlib`, `seaborn`, `jupyterlab`, `nbformat`, `nbconvert`. Full list in `requirements.txt` (a `pip freeze`).
- The published baseline on this dataset reports ≈99% accuracy (Amato et al., 2016); the question we answer here is not "can we match it?" but "does one extra convolution block, holding everything else constant, help?"

---

4. Single-seed results (20 epochs, SEED=42)

Both models trained for 20 epochs on the same data partition. Per-epoch metrics recorded in `reports/metrics_*.csv`; final test-set evaluation captured in notebook cell 17.

4.1 Headline

model	params	test_acc	test_loss	best_val_acc
baseline	5,401,921	**0.9976**	**0.0384**	0.9968
experimental (deeper stack)	3,042,881	0.9849	0.1069	**0.9968**

Baseline wins test accuracy by +1.27 pp and test loss by 0.069. The `best_val_acc` **ties at 0.9968** — so the experimental model *can* reach baseline-level performance; its final-epoch state is much worse (see §4.2 for why: a late-training collapse at epoch 20). Because `train.fit()` evaluates the test set only after the final epoch, the experimental test_acc of 0.9849 reflects the post-collapse model. **The multi-seed comparison in §5 (10 epochs, no collapse artifact) is the cleaner signal and is the load-bearing evidence.**

4.2 Per-epoch reading (verified against `reports/metrics_*.csv`)

metric	baseline	experimental
train_loss ep1 → ep20	0.223 → 0.003	0.206 → 0.004
val_loss: largest jump	ep9→10: 0.032 → 0.042 (+0.010)	**ep19→20: 0.013 → 0.148 (+0.135)**
val_loss: run minimum	ep15 = 0.0219	ep18 = 0.0111
val_acc: run maximum	ep12 = 0.9968 (also ep19)	ep18 = 0.9968 (also ep19)
final (ep20) val_loss / val_acc	0.0227 / 0.9960	**0.1484 / 0.9785**

- Both training losses trend cleanly down through epoch 19 (baseline to 0.003, experimental to 0.001); both reach ~0.998+ train accuracy.
- **Baseline is stable throughout** — val_loss stays in the 0.02–0.04 band from epoch 7 to 20; its biggest single-epoch jump is only +0.010 at ep9→10. Its lowest val_loss (0.0219) is at epoch 15.
- **Experimental is better, then collapses.** Best val_loss 0.0111 at epoch 18 (val_acc 0.997 — matching baseline's best), then a sharp regression at **epoch 20** (val_loss 0.148, val_acc 0.979). The ep19→20 jump (+0.135) is **~13× larger** than baseline's biggest jump (+0.010).
- **The epoch-20 collapse is the headline single-seed finding.** Because `fit()` evaluates the test set only after the final epoch, the experimental test_acc of 0.9849 is measured on the post-collapse model. Its `best_val_acc` (0.9968, tying baseline) is the fairer snapshot of its capability. The deeper architecture exhibits **late-training instability** on this dataset that the shallower baseline does not.

4.3 Confusion matrix and per-class metrics (notebook §7)

- Baseline positive rate: 0.674, true positive rate: 0.999.
- Experimental positive rate: 0.663, true positive rate: 0.981.
- The experimental model's lower true-positive rate (0.981 vs 0.999) reflects the camera-B weakness detailed in §6, not a uniform degradation. The detailed `pd.crosstab` and per-class table are embedded in the notebook.

---

5. Multi-seed paired comparison (notebook §11)

To address the single-seed noise, the same split was held constant while model-init + training-shuffle seeds were varied across {7, 42, 99}. **3 seeds × 2 models × 10 epochs** = 6 training runs. Each pair (baseline vs experimental at the same seed) shares the data partition and therefore the per-seed noise floor; the *paired difference* is the test signal.

5.1 Paired deltas (experimental − baseline) per seed

seed	baseline test_acc	experimental test_acc	delta (pp)
7	0.9952	0.9913	−0.0040
42	0.9873	0.9865	−0.0008
99	0.9944	0.9865	−0.0079
mean	**0.9923**	**0.9881**	**−0.0042**
std	0.0044	0.0028	0.0036

5.2 What this tells us

- **Baseline wins on the multi-seed mean** (0.9923 vs 0.9881, +0.42 pp).
- **Baseline wins all 3 individual seeds** — the deeper model never beat it.
- The deeper model has **lower variance** (std 0.0028 vs 0.0044) — but "consistently worse" is not a win; the narrower band sits below the baseline's band.
- The single-seed +1.27 pp baseline edge in §4 is partly an artifact of the experimental model's epoch-20 collapse; §11 (10 epochs, no collapse) shows the baseline edge is real but smaller (+0.42 pp). The 95% CI on the mean delta at n=3 is roughly ±4 pp (very rough), so we cannot reject "no effect" with high confidence — but the sign is consistent (3/3) and points to baseline.

5.3 The single honest headline

> On CNRPark, the deeper 4-block conv stack did **not help** — it was slightly worse on test accuracy (multi-seed mean −0.42 pp, losing 3/3 seeds), less stable late in training (an epoch-20 val_loss collapse), and weaker on camera B — **despite using ~1.8× fewer parameters** than the 3-block baseline. The shallower model is the better model on this dataset.

This is the strongest claim the new run supports without over-claiming. (An earlier run had favored the experimental model by +0.37 pp; that result was not reproducible and is documented in §7.)

---

6. Second look — per-class by camera (notebook §10)

Domain-shift sanity check: split the 1,258-patch test set by camera (620 from A, 638 from B) and report per-class precision/recall.

6.1 Per-(camera, class) recall (embedded in notebook §10)

model	camera	class	precision	recall	F1	support
baseline	A	free	1.000	1.000	1.000	253
baseline	A	busy	1.000	1.000	1.000	367
baseline	B	free	1.000	0.987	0.991	158
baseline	B	busy	0.996	0.998	0.997	480
experimental	A	free	0.996	0.996	0.996	253
experimental	A	busy	0.997	0.997	0.997	367
experimental	B	free	**0.912**	0.987	**0.948**	158
experimental	B	busy	0.996	**0.969**	0.982	480

6.2 Reading

- Both models are near-perfect on camera A for both classes.
- **The experimental model has a camera-B weakness the baseline does not.** Its `B/free` precision drops to 0.912 (F1 0.948) and its `B/busy` recall drops to 0.969, while baseline holds F1 0.991 / recall 0.998 on the same camera. This is **model-specific and camera-specific** — not a property of camera B alone, because the baseline handles camera B fine.
- The pooled §4 test_acc **partly hid this**: the experimental model's 0.9849 is dragged down disproportionately by camera-B errors. A per-camera *holdout* (train on A, evaluate on B) is the natural next experiment — it would test whether the deeper model generalizes worse to a camera it saw less of, which is the deployable question for P7 (a new site = a new camera).

---

7. Architecture decision history

The experimental model went through three designs before settling on the current depth-only architecture. Preserved for transparency.

variant	what changed vs baseline	measured effect	reason retired
(a) **BatchNorm after every conv**	Add `BatchNorm2d` after each `Conv2d`	+0.24 pp test_acc after 10 epochs (inside noise); +0.066 spike at val_loss ep7	Single-seed noise + the val spike made "did BatchNorm help?" unanswerable from one run
(b) **+ 4th conv block + `AdaptiveAvgPool2d`**	Add a conv block *and* replace	(smoke only)	**Three changes at once** broke the rubric's "one

variant	what changed vs baseline	measured effect	reason retired
	`Flatten` with global avg pool *and* add BN		change" rule
(c) depth only (current)	One extra conv block, same head	baseline +1.27 pp single-seed; **baseline −0.42 pp multi-seed mean (wins 3/3 seeds)**; experimental has late-epoch collapse + camera-B weakness	**Active design** (new run 2026-06-26 reversed an earlier run that had favored experimental by +0.37 pp; that result was not reproducible)

The discarded numbers (a) and (b) appear in earlier notebook revisions; the on-disk `metrics_*.csv` files now reflect (c) only.

---

8. Discussion

8.1 What the data supports

1. **The deeper model trains successfully on this dataset** and reaches competitive *validation* accuracy (best_val_acc 0.9968, tying baseline) — but its final-epoch test accuracy (0.9849) is worse than baseline (0.9976) due to a late-training collapse.
2. **The deeper model has lower parameter count** (~1.8× smaller) — the wider feature map has smaller spatial extent, which dominates the head size. This is the depth-plus-spatial-shrinking pattern O'Shea & Nash (2015) describe as standard CNN practice: a stack of conv blocks each halves spatial resolution while doubling channels, so the parameter count grows sub-linearly with depth. **But smaller did not mean better here.**
3. **The deeper model has lower seed-to-seed variance** (std 0.0028 vs 0.0044) — but consistently lands in a lower accuracy band than baseline.
4. **The deeper model exhibits late-training instability** (epoch-20 val_loss collapse) and a **camera-B weakness** that the baseline does not.

8.2 Limitations and risks

1. **"Depth helps on this dataset" is not supported.** The new run contradicts it: baseline wins single-seed (+1.27 pp), wins multi-seed 3/3 (mean +0.42 pp), and is more stable late in training. (An earlier run had favored experimental by +0.37 pp; that result was not reproducible — see §7.)
2. **The conclusion does not generalize beyond this dataset.** This is one dataset (~12.5k patches, binary, two cameras); other image-classification tasks may show different patterns. A per-camera *holdout* (train on A, evaluate on B) is the natural next experiment and was not run here.

3. **Camera-B generalization is unverified.** §6.2 shows the experimental model has a camera-B weakness the baseline does not (`B/free` precision 0.912, `B/busy` recall 0.969). The pooled test score partly hides this. For deployment (where a new site = a new camera), the per-camera holdout is the load-bearing test, and we have not done it — so neither model is deployment-validated for an unseen camera.
4. **Three seeds is too few for a confidence interval.** The 95% CI on the mean delta at n=3 is roughly ±4 pp, so we cannot reject "no effect" with high confidence; the sign is consistent (3/3 toward baseline) but the magnitude is uncertain. A ≥10-seed sweep (§10.1) is needed before treating the −0.42 pp as a real effect rather than a directional hint.
5. **Single-seed numbers are unreliable on this task.** The single-seed picture (experimental favored by +0.37 pp in an earlier run) was misleading and not reproducible; the multi-seed run reversed its sign. Any single-seed comparison on this dataset carries that risk.
6. **"The deeper model is useless" is also not supported.** Its `best_val_acc` ties baseline (0.9968) — it can reach baseline-level performance. The gap is a training-dynamics + camera-generalization problem, not a capacity problem. A learning-rate schedule, early stopping on val_loss, or BatchNorm might recover the gap (see §10 follow-ups).
7. **No checkpointing on `val_loss`.** `train.fit()` evaluates the test set only after the final epoch, so the experimental model's 0.9849 reflects a post-collapse model rather than its best epoch (ep18, val_acc 0.997). The reported test accuracy therefore understates the deeper model's reachable capability.

8.3 Why this matters for P7

The fusion layer in P7 ingests higher-level events; this notebook demonstrates the PyTorch pattern (Dataset → random_split → DataLoader → model → fit → CSV history → confusion matrix) end-to-end on a binary image-classification task. The transferable lesson for P7 is **methodological**: a controlled single-change comparison is hard to set up in practice (every architectural knob is a tempting target) but pays off in interpretability — the conclusion ("deeper stack did not help on this dataset") survives review because no other knob was turned simultaneously.

A second, more concrete, lesson: **a smaller model is not automatically a better model.** The deeper stack used ~1.8× fewer parameters but was less stable late in training and less accurate on this dataset. For P7, "use the modern smaller backbone" is a hypothesis to test, not a default to assume — and the test must use multiple seeds, because the single-seed picture was misleading in an earlier run.

---

9. Reproducibility

- **Single `SEED = 42`** constant in `src/dataset.py` and `src/train.py`. Both models see the same split indices.
- **Hardware path:** `cnn_env` Python 3.12 venv at `D:/AI_Master/Udacity/capstone_projects/project_04_DeepLearning/cnn_env/` with `torch==2.12.1+cu126`.

- **Reproduce the headline numbers (CLI, no notebook required).** Run from the project root (`D:/AI_Master/Udacity/capstone_projects/project_04_DeepLearning/`):

```
cnn_env/Scripts/python.exe src/train.py \
    --patches-dir data/CNRPark-Patches-150x150 \
    --epochs 20 \
    --out-dir reports
```

- **Reproduce the notebook output (Jupyter).** Run from the project root:

```
cnn_env/Scripts/jupyter lab
```

Open `notebooks/deep_learning.ipynb` and **Kernel → Restart & Run All**. Expected runtime: ~8 minutes for the 20-epoch full run + ~5 minutes for the 3-seed §11 rerun on a GTX 1650 Ti.

- **Dependency set:** `requirements.txt` (a `pip freeze` of `cnn_env`).

---

10. Follow-ups (flagged for future work)

1. **Larger seed sweep (≥10 seeds).** Sharpen the mean-delta estimate and report a proper 95% CI. The current 3-seed sample gives a mean of +0.37 pp with std 0.0056 — too wide to claim a real effect with confidence.
2. **Per-camera *holdout*** (train on A, evaluate on B). §10 already showed that the deeper model has a **camera-B weakness** the baseline does not (B/free F1 0.948, B/busy recall 0.969); a holdout would test the actual domain-shift question for deployment (a new site = a new camera).
3. **BatchNorm + depth (combined).** See §7 history: BatchNorm was tested on a 3-block baseline and depth was tested without BN. A combined model tests whether the two effects stack or cancel.
4. **Weather-stratified evaluation** using `data/CNRPark+EXT.csv` metadata. Tests whether weather (sun, clouds, rain) shifts the model's accuracy.
5. **`tests/test_dataset.py`.** Minimal `pytest` covering `scan_patches` returns the expected (12,584, {busy, free}) shape and a forward-pass shape test for both models.
6. **Checkpointing on val_loss.** `train.fit()` currently only tracks `best_val_acc`. Adding checkpoint-based early stopping on val_loss would let the saved model pick the best-

generalizing epoch (ep18 for experimental — val_loss 0.0111, val_acc 0.997; ep15 for baseline — val_loss 0.0219, val_acc 0.993).

---

11. References

- Amato, G., Bolettieri, P., Carrara, F., Falchi, F., Gennaro, C., & Vairo, C. (2016). **"A System for Counting Free Parking Spaces from Visual Information."** ISTI-CNR technical report. Dataset: <http://cnrpark.it/>.
- Kingma, D. P., & Ba, J. (2015). **"Adam: A Method for Stochastic Optimization."** arXiv:1412.6980 <https://arxiv.org/abs/1412.6980>. (Cited for the Adam optimizer used in §3.4.)
- O'Shea, K., & Nash, R. (2015). **"An Introduction to Convolutional Neural Networks."** arXiv:1511.08458 <https://arxiv.org/abs/1511.08458>. (Cited for the CNN building blocks — conv blocks, max-pooling, the depth-plus-spatial-shrinking pattern — used in the plain-English walkthrough and §3, §8.)
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., et al. (2019). **"PyTorch: An Imperative Style, High-Performance Deep Learning Library."** *Advances in Neural Information Processing Systems 32.* <https://pytorch.org/docs/stable/index.html>. (Cited for the `torch.nn` API, `DataLoader`, `BCEWithLogitsLoss`, and CUDA setup used throughout the implementation.)

---

12. Ethics and responsible use

This experiment processes 150×150 RGB parking-lot patches from the CNRPark dataset, which is released for **research and educational use only** by ISTI-CNR. The images contain **no biometric data** (no faces, no license plates visible at this resolution), **no PII**, and no personally identifying information beyond approximate timestamps and weather conditions already documented in the dataset's metadata.

The trained models are not deployed in any real surveillance system; they exist solely to demonstrate a controlled deep-learning comparison. A production deployment of occupancy classification would require its own evaluation for camera-specific bias (different parking-lot geometry, lighting, and resolution shift the model's inputs) and explicit consent frameworks if applied to people-carrying spaces.

The notebook contains a `## 9. Reproducibility` and a `### Ethics and responsible use` section (cell-27) that mirror the points above for in-notebook reference.